

Answer the questions

Question 1: What is the difference between `int.Parse` and `Convert.ToInt32` when handling null inputs?

- One difference between casting and converting is that converting rounds the double value 9.8 up to 10 instead of trimming the part after the decimal point.

Question 2: What is the difference between `int.Parse` and `Convert.ToInt32` when handling null inputs?

- To avoid errors

Question 3: Explain the real purpose of the `GetHashCode()` method

- `GetHashCode()` generates an integer hash code for an object, mainly used by hash-based collections such as `Dictionary` and `HashSet` to quickly locate and compare objects. It is not a unique identifier.

Question 3: What is the significance of reference equality in .NET?

- Reference equality checks whether two references point to the exact same object in memory, regardless of whether their contents are equal. In .NET

Question 4: Why string is immutable in C#?

- String is immutable in C# for safety, performance, thread safety, and reliable hashing. Any modification creates a new string object instead of changing the original one.

Question 5: How does `StringBuilder` address the inefficiencies of string concatenation?

- **StringBuilder** improves string concatenation performance by using a mutable internal buffer. Instead of creating a new string object for every modification, it modifies the existing buffer, making it more efficient for repeated concatenations.

Question 6: Why is `StringBuilder` faster for large-scale string modifications?

- `StringBuilder` is faster for large-scale string modifications because it is mutable and uses an internal buffer, reducing the need to create new string objects and copy existing content after every modification.

Question 7: Which string formatting method is most used and why?

- **String Interpolation (\$)** is the most commonly used formatting method in modern C# because it is simple, readable, and allows variables and expressions to be embedded directly inside a string.

Question 8: Explain how `StringBuilder` is designed to handle frequent modifications compared to strings.

- **StringBuilder** is designed for frequent string modifications because it is **mutable** and uses an internal expandable buffer. It modifies the existing content instead of creating a new string object for every change, making it more efficient for large or repeated modifications.